

Back of Envelope Calculation in System Design

 systemdesignschool.io/fundamentals/back-of-envelope-calculation



Back-of-the-envelope Resource Estimation

One of the most important aspects of system design is to estimate resources. Given a load parameter such as the number of active users a day, we want to estimate the computing and storage resources we need. Additionally, we want to identify the performance bottlenecks of the service and design our system to address them.

How to Describe Load in System Design

On the product level, this could be Daily Active Users (DAUs). This is commonly translated into engineering parameters such as requests per second.

Requests per second

Commonly used to estimate the storage size requirements such as database and cache memory size

Read/write ratio

- Design the read and write path differently
- If the read to write ratio is high, then common solutions are to use cache to reduce read requests to databases and scale database reads with replications
- If the read-to-write ratio is low, then we want to scale writes using faster data stores with high throughput and partition data stores effectively. We could also consider using eventual consistency to help with greater write performance.

Others

Concurrent web socket connections: Often the bottleneck for [chat app like Whatsapp](#). The solution is to have more servers providing web socket connections.

Load and Performance Parameters

In system design, understanding the load helps engineers optimize resources and build scalable systems. Load can be described using various parameters, which can then be translated into engineering metrics. On a product level, this might involve measuring Daily Active Users (DAU), which can be translated into requests per second for engineering purposes.

Requests Per Second

Requests per second is a common metric used to estimate storage size requirements for databases and cache memory. By calculating the number of requests a system is expected to handle per second, engineers can determine the necessary database size and cache capacity to accommodate the load.

Let's consider an example where we want to calculate the storage requirements for an online content publishing platform. The platform has 500,000 Daily Active Users (DAUs), and we assume that:

- Each user reads 5 blog posts per day on average.
- Each user writes 1 blog post per day on average.
- The average size of a blog post is 50 KB.

First, let's calculate the number of read and write requests per second.

- Total read requests per day = DAUs * average number of blog posts read = $500,000 * 5 = 2,500,000$
- Total write requests per day = DAUs * average number of blog posts written = $500,000 * 1 = 500,000$

Now, let's convert these numbers to requests per second.

- Read requests per second = Total read requests per day / 86,400 seconds (in a day) = $2,500,000 / 86,400 \approx 28.9$
- Write requests per second = Total write requests per day / 86,400 seconds (in a day) = $500,000 / 86,400 \approx 5.8$

Storage Estimation

Now that we have the requests per second, let's calculate the storage size requirements.

For daily storage requirements, we need to consider the write requests:

Daily storage = DAUs * average number of blog posts written * average size of a blog post = $500,000 * 1 * 50 \text{ KB} \approx 25,000,000 \text{ KB}$ or 25 GB

Now, let's assume our retention policy is 10 years and calculate the storage requirements:

Days in 10 years = $365 * 10 = 3,650$

Total storage requirement = Daily storage requirement * days in 10 years = $25 \text{ GB} * 3,650 \approx 91,300 \text{ GB}$ or 91.3 TB

Read/write Ratio and Data Consistency

The read/write ratio is crucial in designing the read and write paths of a system. Depending on the ratio, engineers can optimize read and write operations differently.

High read-to-write ratio: When we have a lot more reads than writes, caching can help reduce read requests to databases. Database reads can be scaled using replication to distribute the read load across multiple instances. For example, a website like Reddit might have a high read-to-write ratio because most users are "lurkers" who read posts and do not create posts themselves. In this case, [caching](#) and [read replicas](#) could be used to ensure smooth operation.

High write-to-read ratio: When we have a lot more writes than reads, engineers should focus on scaling writes using faster data stores with high throughput and partitioning data effectively. Eventual consistency can also help improve write performance. For example, an analytics tool that tracks each user click would have a high write-to-read ratio. It can benefit from partitioning the data store, using high-throughput databases, and implementing eventual consistency (we will cover eventual consistency in the [distributed systems](#) section).

Concurrent connections

Concurrent web socket connections can often be a bottleneck for chat applications like WhatsApp. In this case, engineers could deploy multiple instances of the chat application, each providing web socket connections, and use a load balancer to distribute incoming connections among these instances.

Throughput

Throughput is a measure of the capacity of a system to process, transfer, or handle a specific amount of data, tasks, or transactions within a given period of time. It is a key performance metric used to evaluate the efficiency and effectiveness of a system or network. Throughput can be measured in various units, depending on the context, such as bits per second (bps) for network transfer rates or transactions per second (TPS) for database systems. A higher throughput indicates a system's ability to handle more data, tasks, or transactions in a given time, leading to better performance and user experience.

It's important to note that throughput alone does not tell the whole story. Consider two cases:

Example #1, we have a file storage system that allows users to upload and download large files. Let's assume this system can handle a total of 1000 MB of data transfer per second.

- Each file upload/download request: 100 MB
- Requests per second: 10

The throughput in this case is 1000 MB/s (10 requests * 100 MB/request).

Example #2, consider an API for a mobile app that provides various data points to users, such as weather data, news headlines, and other updates. This API handles small requests and responses but receives a high number of requests per second. Let's assume that each request/response pair has an average size of 1 MB, and the system can handle 1000 MB of data transfer per second.

- Each API request/response: 1 MB
- Requests per second: 1000

The throughput in this case is also 1000 MB/s (1000 requests * 1 MB/request).

Although both systems have the same throughput (1000 MB/s), the requests per second are very different. The file storage system handles 10 requests per second, while the API for the mobile app processes 1,000 requests per second. This demonstrates that throughput and requests per second are related but distinct metrics, as systems with the same throughput can have very different request rates depending on the nature of the requests being served. And we need to design the system according to the **access pattern**.

Latency

In networking, latency is the time it takes for data to travel from one point to another across a network. For example, a packet sent from New York to London may take ~70 ms one way. When you hear about Round Trip Time (RTT), that's simply two one-way latencies combined (there and back).

In system design, latency is defined more broadly: it is the delay between when a client makes a request and when it begins to receive the first byte of the response. This perspective includes:

- Network latency (client → server travel time)
- Queuing delays (waiting in load balancers, queues, or schedulers)
- Initial server processing (the work the server must do before it can start sending back data)

Latency is not the same as response time. Response time includes latency + full server execution + transmission of the entire response payload.

Low latency is critical in real-time applications such as online gaming, video conferencing, live trading, or interactive APIs, where users expect near-instant reactions to their actions.

Response Time

Response time is the total time taken for a system to process a request, from the moment the request is sent until the response is received by the client. It includes *both the latency and the time spent processing the request on the server-side*. In general, users expect fast response times for a smooth user experience, especially for interactive applications like web pages, mobile apps, or gaming platforms.

Note that in many systems (for example, APIs returning small JSON responses or buffered file downloads), the server waits until the whole response is ready before sending anything. In this case, latency and response time collapse into the same measurement because the first and last byte arrive together.

In streaming systems (for example, HTTP chunked responses, video streams, or real-time APIs), the server can send the first byte early while still processing or generating the rest of the response. Here, latency can be much smaller than total response time.

The architecture of large scale systems is highly specific to the application we are building. There is not a one-size-fits-all solution that works in all cases, and we need to design the system considering all the load parameters and access patterns.